View the full repository on GitHub

Code of Practice Quiz App: Annotated Code Walkthrough

This notebook presents the main code modules of the Streamlit-based Code of Practice Quiz app for the IFCS assignment.

About page

This is the code for the About page, which provides a user guide, information about the Code of Practice, and a FAQ section in the app.

In [ ]:
```
import streamlit as st

st.set_page_config(
        page_title="Code of Practice Quiz",
        page_icon="📖"
)

st.title("Code of Practice Quiz")

st.sidebar.image("images/duck_guidance.png")

# --- Table of Contents ---
st.markdown("""
## Table of Contents
- [User Guide](#user-guide)
- [About the Code of Practice](#about-the-code-of-practice)
- [Frequently Asked Questions](#frequently-asked-questions)
""")

# --- User Guide Section ---

st.markdown("## User Guide")
st.markdown("""
Welcome to the Code of Practice Quiz app! This tool helps you test your knowledg
- An interactive quiz to check your understanding
- Information about the Code of Practice
- Frequently asked questions and helpful resources

### How to Take the Quiz
To start the quiz, select the “Test your knowledge” page from the sidebar. Answe

### Where to Find More Information
- Learn more about the Code of Practice in the section below.
- See answers to common questions in the FAQ section.

### Troubleshooting
If you encounter any issues (such as the app not loading or form validation erro

### Privacy Notice
```

```
This app is currently not deployed and is only run locally for demonstration or
""")

# --- About the Code of Practice Section ---
st.markdown("## About the Code of Practice")
st.markdown("""The Code of Practice for Statistics is a set of standards and pri
This applies to anyone from decision-makers and analysts to members of the publi

st.markdown("### What is the Code of Practice?")
st.markdown("""The Code sets out how statistics should be produced and communica

It was created under UK law and maintained by the Office for Statistics Regulati

When producers follow the Code, users can be confident that the statistics are r

While the Code applies formally to official statistics produced by government bo

st.markdown("### Why the Code Matters")
st.markdown("""The Code exists to strengthen trust in statistics by promoting:

-   Trustworthiness – users can be confident in who produced the statistics and
-   Quality – statistics are based on suitable data and sound methods.
-   Value – statistics meet user needs and are accessible and useful.

By following these principles, statistics help inform better decision-making, su

st.markdown("### Who Uses It?")
st.markdown("""The Code is officially required for organisations producing Natio
This includes public bodies, analysts, communicators and anyone who wants to dem

st.markdown("### How to Find Out More")
st.markdown("""If you want to explore the Code in more detail or learn how it’s
-   Visit [the official Code of Practice for Statistics site](https://code.stati
""")

# --- FAQ Section ---
st.markdown("# Frequently Asked Questions")

st.markdown("## Who is this quiz for?")
st.markdown("""This quiz is designed for civil servants especially in the Depart

It can also be used anyone who is interested in data and statistics including:
-   Students and learners exploring statistics
-   Analysts and researchers seeking to test their knowledge
-   Members of the Government Statistical Service (GSS) refreshing their underst

st.markdown("## How does this quiz relate to the Government Statistical Service
st.markdown("""The GSS is the professional network for statisticians across UK g
-   Prepare for GSS assessments
-   Refresh knowledge before official training or CPD activities
-   Understand practical scenarios where the Code applies""")

st.markdown("## What is a “badged” statistician and why does it matter?")
st.markdown("""Within the GSS, some roles require individuals to be “badged”, me
Being badged:
-   Demonstrates professional competence
-   Shows you understand how to apply the Code in real-world contexts
-   May be linked to career progression within the GSS""")
```

Test your knowledge

This script is the main user interface for the quiz. It manages session state, displays questions, collects answers, and shows results.

In [ ]:
```
import streamlit as st
import csv
from utils.export_utils import *
from utils.session_utils import *
from utils.feedback_utils import *
from utils.form_utils import *
import pandas as pd

from utils.quiz_logic import Quiz, User

# Initialize session state variables and load quiz questions from CSV
init_session(quiz = Quiz.from_csv("data/quiz_questions.csv"))

st.title("Test your knowledge")

# Show branding image in sidebar
st.sidebar.image("images/duck_guidance.png")

# Display feedback message if present in session state
if st.session_state.feedback is not None:
    msg_type, msg_content = st.session_state.feedback
    # Show success or error message based on feedback type
    if msg_type == "success":
        st.success(msg_content)
    else:
        st.error(msg_content)
    # Clear feedback after displaying so it doesn't persist
    st.session_state.feedback = None

# If user details are not yet provided, show the user form
if st.session_state.user is None:
    user_form()

# If quiz is ongoing, display the current question and answer options
elif st.session_state.quiz.current_question() is not None:
    q = st.session_state.quiz.current_question()
    st.subheader(q.text)
    # Show possible answers as radio buttons
    answer = st.radio("Choose an answer:", q.possible_answers)
    if st.button("Submit"):
        # Evaluate answer and update feedback in session state
        msg_type, msg_content = give_feedback(st.session_state.quiz, st.session_
        st.session_state.feedback = (msg_type, msg_content)
        # Rerun to update UI and move to next question
        st.rerun()
    # Show current score after each question
    st.write(f"Score: {st.session_state.user.score}")

# If quiz is finished, display results and export options
else:
    st.subheader("Quiz Finished!")
    total = len(st.session_state.quiz.questions)
```

```
    score = st.session_state.user.score
    percent = (score / total) * 100 if total > 0 else 0
    st.write(f"Your final score is {score}/{total}")
    # Show pass/fail message based on score percentage
    if percent >= 80:
        st.success(f"Congratulations! You passed the quiz with {percent:.0f}% co
    else:
        st.error(f"You did not pass. You scored {percent:.0f}%. Try again to imp
    # Save user score to CSV only once
    if not st.session_state.score_saved:
        write_user_scores(st.session_state.user, "user_scores.csv")
    st.session_state.score_saved = True
    # Prepare CSV data for download
    csv_data = export_results(st.session_state.user)
    st.download_button(
        label = "Export my answers",
        data = csv_data,
        file_name = "COP_quiz_answers.csv",
        mime = "text/csv"
    )
    # Allow user to restart the quiz by clearing session state
    if st.button("Restart Quiz"):
        del st.session_state.quiz
        del st.session_state.user
        st.rerun()
```

Contact us

In [ ]:

```
import streamlit as st

# Set page configuration for title and icon
st.set_page_config(page_title="Contact Us", page_icon="")

# Main page title
st.title("Contact Us")

# Show branding image in sidebar
st.sidebar.image("images/duck_guidance.png")

# Display contact instructions and support email
st.markdown("""
If you have questions, feedback, or need support regarding the Code of Practice

**Email:** [statistics.development@education.gov.uk](mailto:statistics.developme

We aim to respond within 3 working days.

""")
```

Utils code

Export utils

In [ ]:

```
import csv
import streamlit as st
```

```python
import io

def write_user_scores(user, file_name):
    """
    Append a user's score and details to a CSV file.
    Args:
        user (User): The user whose score is being recorded.
        file_name (str): The path to the CSV file.
    """
    try:
        # Open the CSV file in append mode
        with open(file_name, "a", newline="") as csvfile:
            writer = csv.DictWriter(csvfile, fieldnames=["first_name", "last_nam
            # Write header if file is empty
            if csvfile.tell() == 0:
                writer.writeheader()
            # Write user details and score as a new row
            writer.writerow({
                "first_name": user.first_name,
                "last_name": user.last_name,
                "email": user.email,
                "score": user.score
            })
    except Exception as e:
        # Show error in Streamlit if writing fails
        st.error(f"An unexpected error occurred: {e}")




def export_results(user):
    """
    Export a user's quiz answers as a CSV string.
    Args:
        user (User): The user whose answers are being exported.
    Returns:
        str: CSV data as a string.
    """
    # Try to export user answers to CSV string, handle errors gracefully
    try:
        export_rows = []
        # Loop through each answer and prepare row for CSV
        for ans in user.answers:
            export_rows.append({
                "question": ans["question"].text,  # Get question text
                "aspect": ans["question"].aspect,  # Get question aspect/categor
                "your_answer": ans["answer"],      # User's answer
                "correct": ans["correct"]           # Whether answer was correct
            })
        # Create CSV in memory using StringIO
        output = io.StringIO()
        writer = csv.DictWriter(output, fieldnames=["question", "aspect", "your_
        writer.writeheader()
        writer.writerows(export_rows)
        # Return CSV data as string
        return output.getvalue()
    except Exception as e:
        # Return empty string if export fails
        return ""
```

Feedback utils

In [ ]:
```
import streamlit as st

def give_feedback(quiz, user, answer):
    """
    Evaluate the user's answer, update their record, and return feedback message
    Args:
        quiz (Quiz): The current quiz instance.
        user (User): The user taking the quiz.
        answer (str): The answer provided by the user.
    Returns:
        tuple: (str, str) feedback type and message.
    """
    try:
        # Check if the answer is correct and get the current question
        correct, question = quiz.answer_current(answer)
        # Record the user's answer and update score if correct
        user.record_answer(question, answer, correct)
        if correct:
            # Return success feedback if answer is correct
            return ("success", "Correct! 🎉")
        else:
            # Return error feedback and show correct answer if wrong
            return ("error", f"Incorrect! The correct answer was: {question.corr
    except Exception as e:
        # Show error in Streamlit if feedback fails
        st.error(f"Loading feedback failed: {e}")
        return ("error", "An unexpected error occurred while processing your ans
```

Form utils

In [ ]:
```
import streamlit as st
import re
from utils.quiz_logic import User


def validate_name(name):
    """
    Validate that the provided name is at least 2 characters and alphanumeric.
    Args:
        name (str): The name to validate.
    Returns:
        bool: True if valid, False otherwise.
    """
    # Name must be at least 2 characters
    if len(name) <= 1:
        return False
    # Name must be alphanumeric (no special chars)
    if not name.isalnum():
        return False
    return True


def validate_email(email):
    """
```

```
    Validate that the provided email matches a standard email pattern.
    Args:
        email (str): The email address to validate.
    Returns:
        bool: True if valid, False otherwise.
    """
    # Regex for basic email validation
    regex = r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,7}"
    if re.fullmatch(regex, email):
        return True
    return False


def user_form():
    """
    Display a Streamlit form for user details and validate input.
    On successful submission, stores a User object in session state.
    Returns:
        None
    """
    # Display form for user input
    with st.form("user_details"):
        first = st.text_input("First Name")
        last = st.text_input("Last Name")
        email = st.text_input("Email")
        submitted = st.form_submit_button("Start Quiz")
        if submitted:
            # Check all fields are filled
            if not (first and last and email):
                st.warning("Please fill in all fields.")
            # Validate first name
            elif not (validate_name(first)):
                st.warning("Please enter a valid first name")
            # Validate last name
            elif not (validate_name(last)):
                st.warning("Please enter a valid last name")
            # Validate email format
            elif not(validate_email(email)):
                st.warning("Please enter a valid email")
            else:
                try:
                    # Create User object and store in session state
                    st.session_state.user = User(first, last, email)
                    # Rerun to update UI and proceed to quiz
                    st.rerun()
                except Exception as e:
                    # Show error if user creation fails
                    st.error(f"An unexpected error occurred: {e}")
```

Quiz logic

In [ ]:

```
import pandas as pd


class Question:
    """
    Represents a quiz question with possible answers and the correct answer.
```

```python
    Attributes:
        text (str): The question text.
        possible_answers (list): List of possible answers.
        correct_answer (str): The correct answer.
        aspect (str): The aspect/category of the question.
    """
    def __init__(self, text, possible_answers, correct_answer, aspect):
        """
        Initialize a Question instance.
        Args:
            text (str): The question text.
            possible_answers (list): List of possible answers.
            correct_answer (str): The correct answer.
            aspect (str): The aspect/category of the question.
        """
        self.text = text  # Question text
        self.possible_answers = possible_answers  # List of possible answers
        self.correct_answer = correct_answer  # The correct answer
        self.aspect = aspect  # Category/aspect of the question

    def is_correct(self, answer):
        """
        Check if the provided answer is correct.
        Args:
            answer (str): The answer to check.
        Returns:
            bool: True if correct, False otherwise.
        """
        # Compare provided answer to correct answer
        return answer == self.correct_answer



class User:
    """
    Represents a user taking the quiz, tracking their details and answers.
    Attributes:
        first_name (str): User's first name.
        last_name (str): User's last name.
        email (str): User's email address.
        score (int): User's current score.
        answers (list): List of answer records.
    """
    def __init__(self, first_name, last_name, email):
        """
        Initialize a User instance.
        Args:
            first_name (str): User's first name.
            last_name (str): User's last name.
            email (str): User's email address.
        """
        self.first_name = first_name  # User's first name
        self.last_name = last_name    # User's last name
        self.email = email            # User's email address
        self.score = 0               # User's quiz score
        self.answers = []            # List of user's answers


    def record_answer(self, question, answer, correct):
        """
```

```
        Record an answer for the user and update score if correct.
        Args:
            question (Question): The question answered.
            answer (str): The user's answer.
            correct (bool): Whether the answer was correct.
        """
        # Store answer record (question object, answer, correctness)
        self.answers.append({"question": question,
                             "answer": answer,
                             "correct": correct})
        # Increment score if answer was correct
        if correct:
            self.score += 1


class Quiz:
    """
    Manages the quiz, including questions, current state, and answer logic.
    """
    def __init__(self, questions):
        """
        Initialize a Quiz instance.
        Args:
            questions (list): List of Question objects.
        """
        self.questions = questions      # List of Question objects
        self.current_index = 0          # Index of current question


    @classmethod
    def from_csv(cls, csv_path):
        """
        Create a Quiz instance from a CSV file of questions.
        Args:
            csv_path (str): Path to the CSV file.
        Returns:
            Quiz: An instance of the Quiz class.
        """
        try:
            df = pd.read_csv(csv_path)
            questions = []
            # Group by unique question text to build Question objects
            for q_text in df["question"].unique():
                q_df = df[df["question"] == q_text]
                possible_answers = list(q_df["possible_answers"])
                # Find the correct answer using answer_indicator column
                correct_answer = q_df[q_df["answer_indicator"] == "c"]['possible
                aspect = q_df["aspect"].iloc[0]
                questions.append(Question(q_text, possible_answers, correct_answ
            return cls(questions)
        except Exception as e:
            # Raise error if CSV loading fails
            raise RuntimeError(f"Failed to load quiz CSV: {e}")

    def current_question(self):
        """
        Get the current question in the quiz.
        Returns:
            Question or None: The current Question object or None if finished.
        """
```

```
        # Return current question if quiz not finished
        if self.current_index < len(self.questions):
            return self.questions[self.current_index]
        else:
            return None

    def answer_current(self, answer):
        """
        Answer the current question and advance the quiz.
        Args:
            answer (str): The answer provided by the user.
        Returns:
            tuple: (bool, Question) indicating if correct and the question objec
        """
        question = self.current_question()
        # Check if answer is correct
        correct = question.is_correct(answer)
        # Move to next question
        self.current_index += 1
        return correct, question
```

Session utils

In [ ]:

```
import streamlit as st


def init_session(quiz = None):
    """
    Initialize Streamlit session state variables for the quiz application.
    Args:
        quiz (Quiz, optional): The quiz instance to store in session state.
    Returns:
        None
    """
    # Add quiz object to session state if not already present
    if "quiz" not in st.session_state and quiz is not None:
        st.session_state.quiz = quiz
    # Initialize feedback message state
    if "feedback" not in st.session_state:
        st.session_state.feedback = None
    # Initialize user object state
    if "user" not in st.session_state:
        st.session_state.user = None
    # Track if score has been saved to CSV
    if "score_saved" not in st.session_state:
        st.session_state.score_saved = False
    # Store exported results for download
    if "result_export" not in st.session_state:
        st.session_state.result_export = None
```

Tests

Testing export utils

In [ ]:
```
import tempfile
import csv
import os
from app.utils.quiz_logic import User, Question
from app.utils.export_utils import write_user_scores, export_results

def test_write_user_scores():
    """
    Test that write_user_scores correctly writes a user's score to a CSV file.
    """
    # Create a test user
    user = User("Test", "User", "test@example.com")
    user.score = 5

    # Create a temporary file
    with tempfile.NamedTemporaryFile(delete=False, mode='w+', newline='') as tmp
        tmpfile_name = tmpfile.name

    # Patch the function to write to the temp file instead of the real one

    write_user_scores(user, file_name=tmpfile_name)

    # Read back the file and check contents
    with open(tmpfile_name, newline='') as csvfile:
        reader = csv.DictReader(csvfile)
        rows = list(reader)
        assert len(rows) == 1
        assert rows[0]["first_name"] == "Test"
        assert rows[0]["last_name"] == "User"
        assert rows[0]["email"] == "test@example.com"
        assert rows[0]["score"] == "5"

    # Clean up
    os.remove(tmpfile_name)


def test_export_results():
    """
    Test that export_results returns a CSV string containing the user's answers.
    """
    user = User("Test", "User", "test@example.com")
    q1 = Question(
        text="What is 1+1?",
        possible_answers=["1", "2", "3", "4"],
        correct_answer="2",
        aspect="maths"
    )
    q2 = Question(
        text="What is 2+2?",
        possible_answers=["1", "2", "3", "4"],
        correct_answer="4",
        aspect="maths"
    )

    user.record_answer(q1, "2", True)
    user.record_answer(q1, "3", False)

    csv_data = export_results(user)
```

```
    assert "What is 1+1?" in csv_data
    assert "True" in csv_data
```

Testing quiz logic

In [ ]:
```
import pytest
from app.utils.quiz_logic import *

# test different classes in the quiz logic work as expected

#testing the question class

def test_question_is_correct():
    """
    Test that Question.is_correct returns True for the correct answer and False
    """
    q = Question(
        text = "What is 1+1?",
        possible_answers = ["1","2","3","4"],
        correct_answer = "2",
        aspect = "maths"
    )

    assert q.is_correct("2") is True
    assert q.is_correct("4") is False

#test user class

def test_user_record_answer():
    """
    Test that User.record_answer correctly records answers and updates the score
    """
    user_info = User(
        first_name = "John",
        last_name = "Doe",
        email = "john.doe@gmail.com"
    )

    q = Question(
        text="What is 1+1?",
        possible_answers=["1", "2", "3", "4"],
        correct_answer="2",
        aspect="maths"
    )

    #test for correct answers
    user_info.record_answer(q,"2",True)
    assert user_info.score == 1
    assert user_info.answers[-1]["answer"]=="2"
    assert user_info.answers[-1]["correct"] is True

    # test for incorrect answers
    user_info.record_answer(q,"1", False)
    assert user_info.score == 1
    assert user_info.answers[-1]["answer"] == "1"
    assert user_info.answers[-1]["correct"] is False
```

```
def test_quiz_from_csv():
    """
    Test that Quiz.from_csv loads questions from a CSV file and returns a Quiz i
    """
    csv_path = "data/quiz_questions.csv"

    result=Quiz.from_csv(csv_path)

    assert isinstance(result, Quiz)
    assert len(result.questions)>0


def test_quiz_current_question():
    """
    Test that Quiz.current_question returns the correct question based on curren
    """
    csv_path = "data/quiz_questions.csv"

    quiz=Quiz.from_csv(csv_path)
    quiz.current_index= 5
    expected_question = quiz.questions[5]
    assert quiz.current_question() == expected_question



def test_answer_current():
    """
    Test that Quiz.answer_current returns correct results and advances the quiz
    """
    quiz = Quiz.from_csv("data/quiz_questions.csv")
    quiz.current_index = 0
    question = quiz.current_question()
    correct_answer = question.correct_answer
    wrong_answer = [a for a in question.possible_answers if a != correct_answer]

    # Test correct answer
    was_correct, returned_question = quiz.answer_current(correct_answer)
    assert was_correct is True
    assert returned_question == question
    assert quiz.current_index == 1

    # Test incorrect answer
    question2 = quiz.current_question()
    was_correct2, returned_question2 = quiz.answer_current(wrong_answer)
    assert was_correct2 is False
    assert returned_question2 == question2
    assert quiz.current_index == 2
```

Testing session utils

In [ ]:

```
import pytest
from app.utils.session_utils import *
import streamlit as st

def test_init_session_no_quiz():
    """
```

```
    Test that init_session initializes all required session state variables when
    """
    init_session(quiz = None)


    assert st.session_state.feedback == None
    assert st.session_state.user == None
    assert st.session_state.score_saved == False
    assert st.session_state.result_export == None
```

init.py files

These files are used to mark directories as Python packages. They are often empty but are required for package/module imports.

In [ ]:
```
# app/utils/__init__.py
# tests/__init__.py
# (Empty files, used to mark directories as Python packages)
```